

B² Club — Application

Everything about this application: what it does, how it is built, where it runs, what every environment variable is for, and how to deploy it.

Written for a developer who has not worked on this codebase before. If you are picking this up cold, read §1 and §2, then jump to whatever you need.

****Contents****

1. [What the application is](#1-what-the-application-is)
2. [Architecture and hosting](#2-architecture-and-hosting)
3. [Environment variables](#3-environment-variables)
4. [The database](#4-the-database)
5. [Functionality, feature by feature](#5-functionality-feature-by-feature)
6. [API reference](#6-api-reference)
7. [Scheduled jobs](#7-scheduled-jobs)
8. [File storage](#8-file-storage)
9. [Email](#9-email)
10. [Payments](#10-payments)
11. [Deployment](#11-deployment)
12. [Local development](#12-local-development)
13. [Testing](#13-testing)

1. What the application is

A membership and event platform for B² Club, a running club in Madurai, India. Members sign up, register for club events (runs, treks, badminton, socials), pay through Razorpay, and receive a QR ticket that is scanned at the start line. Organisers create events, manage who is coming, take payments, and export rosters to build the event's WhatsApp group.

There are four roles:

Role	What it can do
------	----------------

---	---
-----	-----

<code>`VISITOR`</code>	Browse published events, the gallery, polls and the about page
------------------------	--

<code>`MEMBER`</code>	Everything above, plus register for events, hold a ticket, post in the forum, vote in polls, submit results and feedback
-----------------------	--

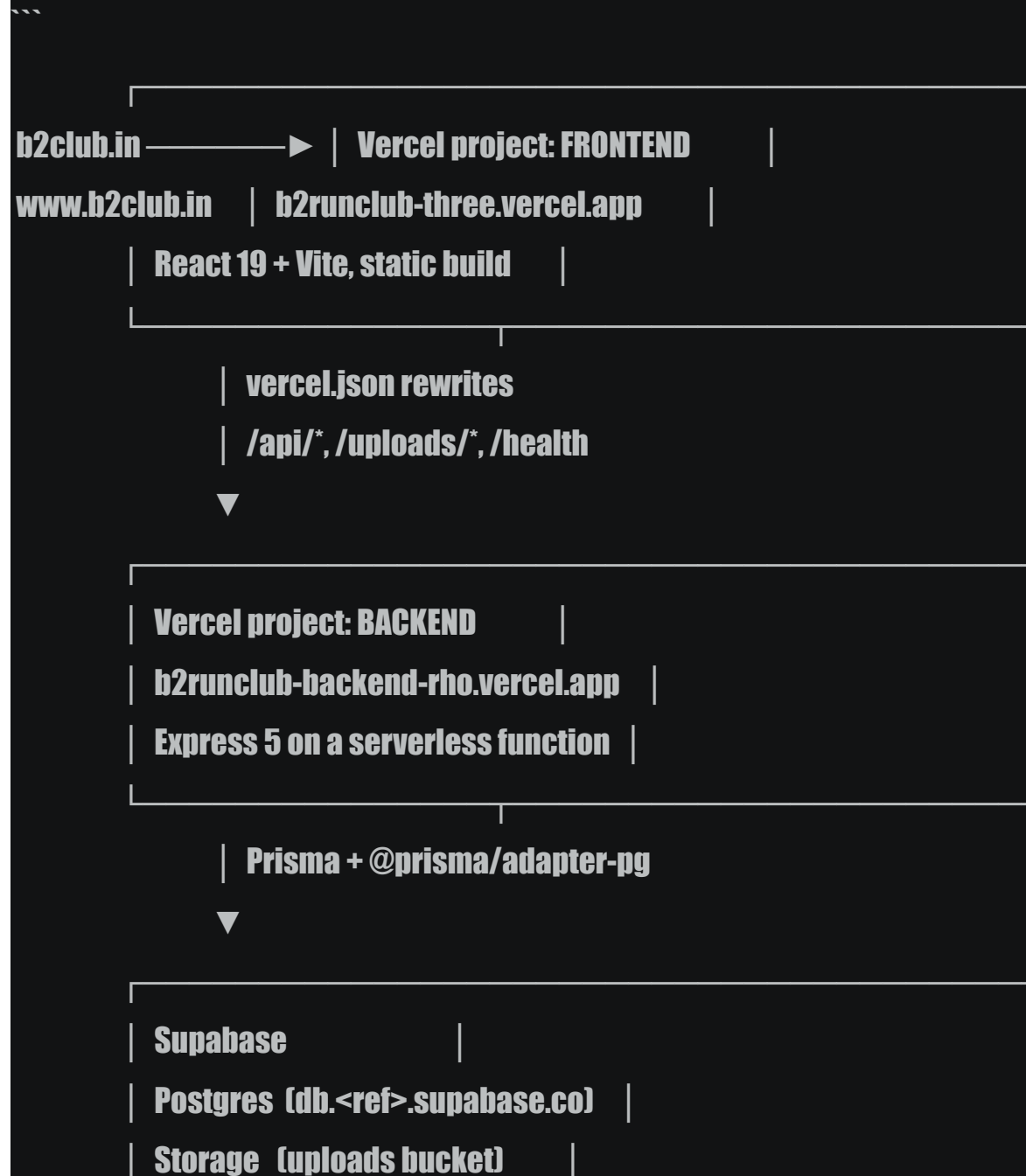
<code>`VOLUNTEER`</code>	Everything a member can, plus their own event entry is free (guests they bring still pay); claims marshal shifts and runs check-in
--------------------------	--

<code>`ADMIN`</code>	Everything: create and publish events, manage registrations and members, take refunds, run exports, browse the database
----------------------	---

Roles are plain strings on ``User.role``, not a database enum.

2. Architecture and hosting

Three services, all managed, no servers to maintain.



External: Razorpay (payments) · Gmail SMTP (email)

Why the frontend proxies the API

The frontend's `(runclub-frontend/vercel.json)` rewrites `/api/``, `/uploads/`` and `/health`` to the backend project. This means the browser only ever talks to one origin, so **there is no CORS to configure** and no `VITE_API_BASE_URL`` to keep in step with a moving backend URL. The last rewrite (`/(.*)` → /index.html``) is the SPA fallback that makes client-side routes like `/events/abc`` work on a hard refresh.

If you rename or redeploy the backend to a different URL, **the three rewrite destinations in the frontend's `vercel.json`` must be updated**. That file is the only place the backend hostname is written down.

The backend is one serverless function

`(runclub-backend/api/index.ts)` re-exports the Express app, and `vercel.json`` rewrites every path to `/api``. So Express does the routing, and Vercel invokes it per request.

Consequences you must know about:

- **`app.listen()` never runs.** `server.ts` checks for `process.env.VERCEL` and skips it. Locally it listens on `PORT` (default 3000).
- **`setInterval` does not survive.** The function is frozen the moment it responds, so the in-process reminder and hold schedulers never fire in production. Vercel Cron calls HTTP endpoints instead — see §7.
- **The filesystem is read-only** apart from `/tmp`, which is not shared between invocations. Uploads therefore have to go to object storage — see §8.
- **`npm start` is never executed.** Migrations and the seed run in the *build* command instead. See §11.

Tech stack

|||

|---|---|

| Frontend | React 19, Vite 6, TypeScript, Tailwind 4, React Router, Framer Motion, GSAP, three.js / react-three-fiber, jsQR (ticket scanning) |

| Backend | Node, Express 5, TypeScript, Prisma 7 |

| Database | Postgres (Supabase), via `@prisma/adapters-pg` |

| Auth | JWT (`jsonwebtoken`), signed with `JWT_SECRET` |

| Payments | Razorpay Checkout + webhook |

| Email | Nodemailer over Gmail SMTP |

| Spreadsheets | ExcelJS (server-side `.xlsx`) |

| QR | `qrcode` (generate), `jsqr` (scan in browser) |

3. Environment variables

Backend

Set these in ****Vercel → backend project → Settings → Environment Variables****.
Locally they live in ``runclub-backend/.env``.

Required — the app refuses to start without them

Key	What it is for	Notes
-----	----------------	-------

---	---	---
-----	-----	-----

<code>`DATABASE_URL`</code>	Postgres connection string the app queries through	**Must be the Supabase connection pooler, port 6543.** Serverless opens a fresh connection per invocation, and a direct connection exhausts Postgres's limit almost immediately
-----------------------------	--	--

<code>`JWT_SECRET`</code>	Signs and verifies every session token	Validated at startup: at least 24 characters, and refused if it matches a known example value. Rotating it logs everybody out
---------------------------	--	---

Required in practice

Key	What it is for	Notes
-----	----------------	-------

---	---	---
-----	-----	-----

| ``DIRECT_URL`` | Postgres connection used by ****migrations only****, port 5432 | The pooler runs in transaction mode, which does not support the prepared statements and session state ``prisma migrate`` needs. Read by `[prisma.config.ts](runclub-backend/prisma.config.ts)` |

| ``APP_URL`` | Public frontend URL, used to build links inside emails | e.g. ``https://b2club.in``. Wrong value = dead links in every email |

Payments — Razorpay

| Key | What it is for | Notes |

|---|---|---|

| ``RAZORPAY_KEY_ID`` | Publishable key id | ``rzp_live_...`` in production, ``rzp_test_...`` for staging. Also served to the browser by ``GET /api/payments/config``, which is safe — key ids are public |

| ``RAZORPAY_KEY_SECRET`` | Creates orders, verifies Checkout callbacks, issues refunds | Never leaves the server |

| ``RAZORPAY_WEBHOOK_SECRET`` | Verifies the signature on incoming webhooks | Must match the secret entered in the Razorpay dashboard. ****With this unset the webhook endpoint rejects every request with 503**** rather than trusting unsigned calls |

With ``RAZORPAY_KEY_ID`` or ``RAZORPAY_KEY_SECRET`` missing, the app runs in ****mock mode****: it mints fake ``order_mock_...`` ids and exposes a dev-only ``/simulate`` route. Useful locally, never in production.

Email — Gmail SMTP

Key	What it is for	Notes
-----	----------------	-------

---	---	---
-----	-----	-----

<code>`SMTP_HOST`</code>	<code>`smtp.gmail.com`</code>	
--------------------------	-------------------------------	--

<code>`SMTP_PORT`</code>	<code>`587`</code> (STARTTLS) or <code>`465`</code> (implicit TLS)	The code treats 465 as secure and everything else as STARTTLS
--------------------------	--	---

<code>`SMTP_USER`</code>	The club Gmail address	
--------------------------	------------------------	--

<code>`SMTP_PASS`</code>	A **16-character Gmail App Password** , not the account password Gmail refuses account passwords over SMTP, and **Invalidates every App Password when the account password is changed** . See §9
--------------------------	--

<code>`MAIL_FROM`</code>	From address on outgoing mail Must be the same address you authenticate as; Gmail refuses to send as an unrelated domain
--------------------------	--

With any of ``SMTP_HOST`` / ``SMTP_USER`` / ``SMTP_PASS`` missing, the mailer falls back to logging emails to the console instead of sending them.

Scheduled Jobs

Key	What it is for	Notes
-----	----------------	-------

---	---	---
-----	-----	-----

<code>`CRON_SECRET`</code>	Authenticates calls to <code>`/api/cron/*`</code> **Unset disables both cron endpoints (503)** , because they send real email. Accepted as <code>`Authorization: Bearer <secret>`</code> or <code>`?key=<secret>`</code>
----------------------------	---

First admin account (seed)

Key	What it is for	Notes
-----	----------------	-------

---	---	---
-----	-----	-----

<code>`ADMIN_EMAIL`</code>	Email of the admin account to create on deploy	
----------------------------	--	--

<code>`ADMIN_PASSWORD`</code>	Its password	Must be at least 8 characters or the seed skips
-------------------------------	--------------	---

<code>`ADMIN_NAME`</code>	Display name	Optional, defaults to "Club Admin"
---------------------------	--------------	------------------------------------

The seed is safe to run repeatedly: it exits early if any ADMIN already exists, and promotes a matching existing MEMBER rather than creating a duplicate.

File storage — pick one set

Key	What it is for
-----	----------------

---	---
-----	-----

<code>`SUPABASE_URL`</code>	Supabase project URL. Preferred driver
-----------------------------	--

<code>`SUPABASE_SERVICE_ROLE_KEY`</code>	**Service role** key, not the anon key — uploads are server-side and must bypass RLS
--	---

<code>`SUPABASE_BUCKET`</code>	Bucket name. Optional, defaults to <code>`uploads`</code>
--------------------------------	---

<code>`BLOB_READ_WRITE_TOKEN`</code>	Vercel Blob token. Used only if the Supabase pair is absent. Vercel injects it automatically when a Blob store is attached
--------------------------------------	--

<code>`UPLOAD_DIR`</code>	Local disk path. The fallback driver, for local development only
---------------------------	--

See §8 for how the driver is chosen.

Optional

Key	What it is for
-----	----------------

---	---
-----	-----

<code>`PORT`</code>	Local listen port. Default 3000. Ignored on Vercel
---------------------	--

<code>`NODE_ENV`</code>	<code>`production`</code> disables the dev-only <code>`/api/payments/simulate`</code> route
-------------------------	---

--	--

<code>`DATABASE_AUTH_TOKEN`</code>	Only for a remote libSQL host (Turso). Ignored by Postgres and by <code>`file:`</code> URLs
------------------------------------	---

<code>`API_URL`</code>	Present in the env file but not read anywhere in the code
------------------------	---

Leftovers that can be deleted

``STRAVA_CLIENT_ID``, ``STRAVA_CLIENT_SECRET``, ``STRAVA_REDIRECT_URI`` — the Strava

integration was removed (migrations ``4_drop_strava`` and ``5_strava_links``). No code reads these. Safe to remove from every environment.

Frontend

Key	What it is for	Notes
-----	----------------	-------

---	---	---
-----	-----	-----

| ``VITE_API_BASE_URL`` | Base URL for API calls | ****Leave unset.**** Empty means same-origin, which is what the ``vercel.json`` rewrites provide. Setting it reintroduces CORS |

| ``VITE_RAZORPAY_KEY_ID`` | Publishable key, a fallback for resuming a payment | Optional and largely redundant — the app prefers the key from ``GET /api/payments/config``, which cannot drift from the secret it pairs with. This variable was unset on the hosted frontend once and broke resumed payments, which is why config is preferred |

``VITE_``-prefixed variables are ****compiled into the JavaScript bundle**** and are readable by anyone. Never put a secret in one.

4. The database

Supabase Postgres, 27 models. Managed with Prisma; migrations are hand-written SQL under `[prisma/migrations](runclub-backend/prisma/migrations)`.

Two connection strings, and why

	Port	Used by	Why
--	------	---------	-----

---	---	---	---
-----	-----	-----	-----

<code>`DATABASE_URL`</code> 6543 (pooler) The application Serverless opens a connection per invocation; the pooler multiplexes them

| ``DIRECT_URL`` | 5432 (direct) | ``prisma migrate`` | The pooler's transaction mode does not support prepared statements or session state |

Getting this backwards produces either connection-limit errors under load (app on 5432) or migration failures (migrations on 6543).

Core models

****People and access****

- ``User`` — email, password hash, name, role, own mobile (``phone``, E.164), ``emergency_contact``, ``email_verified_at``
- ``VerificationCode`` — one-time email codes
- ``PasswordResetToken`` — single-use reset links

****Events****

- ``Event`` — title, discipline, ``date_time`` (UTC instant), location, price, status (``DRAFT`` / ``PUBLISHED`` / ``ARCHIVED``), capacity, ``kids_allowed``, ``kid_price``, ``party_discount``, ``discount_min_party``, ``hold_minutes``, cover image, GPX route
- ``EventQuestion`` / ``RegistrationAnswer`` — the per-event questionnaire
- ``EventReminder`` / ``ReminderDelivery`` — scheduled reminder emails and a per-recipient delivery log that makes sends idempotent

****Bookings****

- ``EventRegistration`` — one booking, which may cover several people. Carries

status, Razorpay order and payment ids, a price snapshot

(`amount\_due\_paise`, `adult\_price\_at\_booking`, `kid\_price\_at\_booking`,
`discount\_paise\_at\_booking`), the hold deadline (`hold\_expires\_at`) and the
lifecycle stamps (`expired\_at`, `cancelled\_at`, `cancel\_reason`)

- `RegistrationGuest` — **one row per participant**, including the booker.

Name, `kind` (`ADULT` / `KID`), `is\_booker`, `phone`, `admitted\_at`

****Race day****

- `EventShift` / `ShiftAssignment` — marshal shifts

- `Checkpoint` / `CheckpointSplit` — on-course timing

****After the event****

- `EventResult`, `EventFeedback`, `Photo`, `HealthWorkout`

****Community and content****

- `Post`, `Comment`, `Poll`, `PollOption`, `PollVote`, `Notification`,

`ClubInfo`, `Collaborator`, `Founder`

Registration statuses

Defined in

(`utils/registrationStatus.ts` ([runclub-backend/src/utils/registrationStatus.ts](#)),

which also holds the predicates (`isTicketed`, `isClosed`, `shouldReceiveMail`)

that every route asks rather than comparing strings inline.

	Stored	Shown to people	Holds a place?	Ticket?
	---	---	---	---
`PENDING`	Awaiting Payment	Yes, until `hold_expires_at`	No	
`PAID`	Paid	Yes	Yes	
`FREE`	Free	Yes	Yes	
`FAILED`	Payment Failed	No	No	
`EXPIRED`	Spot Expired – Register Again	No	No	
`CANCELLED`	Cancelled	No	No	
`DEACTIVATED`	Deactivated	No	No	
`TEST`	Test Registration	No	No	

`PENDING` is deliberately ****not**** renamed to **`AWAITING\_PAYMENT`**: the display name is applied in the UI, because renaming would mean rewriting live rows for something only ever seen as a label.

Only **`CANCELLED`**, **`DEACTIVATED`** and **`TEST`** can be set by an admin. Payment states belong to Razorpay and the booking flow; an admin cannot type "PAID" over an unpaid booking.

Timezone

****Every timestamp is stored as a UTC Instant.**** The club's timezone (**`Asia/Kolkata`**, UTC+5:30) is applied only when rendering, via **utils/time.ts**(**runclub-backend/src/utils/time.ts**).

This matters because Vercel runs functions with `TZ=UTC`. A bare `Intl.DateTimeFormat("en-IN", ...)` sets the *language*, not the zone, so it renders in UTC — which is how a 6:00 AM event once went out in a reminder email as "12:30 am". Anything formatted server-side must pass `timeZone`, and the helpers in `time.ts` do. Do not "fix" this by storing local times; that would corrupt every existing row.

5. Functionality, feature by feature

Accounts

- **Sign up** with name, email, password (minimum 8 characters, enforced server-side) and mobile number. The mobile is mandatory and normalised to E.164 — `98765 43210` is stored as `+919876543210`. An explicitly dialled `+<cc>` is kept as given, so an overseas member is not rewritten to +91.
- **Email verification** by one-time code. Unverified members can browse but **cannot register for an event**.
- **Login** issues a JWT held in `localStorage`. Password reset by emailed single-use link. Change email (re-verifies), change password, change mobile and emergency contact from the profile page.
- Every password field has a show/hide eye toggle, hidden by default.

Events

Organisers create events as drafts and publish when ready. Each event has a discipline (a preset or free text), date and time, location, entry fee, optional capacity, optional cover image and optional GPX route.

Pricing options:

- ****Child pricing**** — `kids_allowed` plus a `kid_price`. A child cannot be added if the organiser has not set a price; refusing is more honest than guessing at zero.
- ****Group discount**** — a flat rupee amount off the whole booking, once, applied from a configurable party size (`discount_min_party`, default 2). Clamped so it can never exceed the total or turn into a surcharge.
- ****Payment hold**** — `hold_minutes`, default 24 hours. See below.

Registration

One booking can cover up to ****6 people**** including the member. For each:

- Name, and whether they are an adult or a child
- ****A mobile number each**** — the numbers the club adds to the event WhatsApp group. A "use my number" checkbox copies the booker's, for a child or partner with no number of their own. It is sent as a flag and resolved server-side, so a tampered request cannot attach an arbitrary number while claiming it is the booker's
- The booker supplies their own ****WhatsApp number**** and a separate ****emergency**

contact^{**}. The two must differ — an emergency contact that rings the casualty's own phone is worse than useless. Both are saved back to the account, so the next booking is prefilled

- Answers to the event's **questionnaire**^{**}, if it has one
- A **liability waiver**^{**}, whose wording changes when the booking covers other people, including children

Capacity counts **people, not bookings**^{**} — a party of four uses four places. A volunteer's own place is exempt; anyone they bring is not.

The 24-hour spot hold

An unpaid booking holds its places until `hold_expires_at`, then releases them.

The mechanism worth understanding: **a lapsed hold is treated as released at read time**^{**}, by `seatFilter()` in

`events.router.ts` (`runclub-backend/src/routes/events.router.ts`), not only when the sweep runs. Vercel's Hobby plan allows one cron run per day, which would otherwise make a "24 hour" hold last anywhere from 24 to 48 hours. Deciding it at read time makes availability correct the instant the deadline passes, whatever the scheduler is doing. The sweep then only writes the durable record and notifies the member.

An expired booking is kept, not deleted, and the member may register again.

Payments

Razorpay Checkout opens over the app. See §10 for the full flow and the failure mode it was built to survive.

Tickets and check-In

A paid or free booking gets ****one QR ticket covering the whole party****, served as an HTML page listing every name. Blocked members are refused a ticket even though their payment status is untouched.

At the start line a marshal scans with the device camera (jsQR). A party can be ****admitted person by person****, so a family of four can be half checked in. An admission can be undone.

Organiser tools

- ****Roster**** — every booking for an event, with status, party, and ****a mobile number against every participant****, visible without expanding anything
- ****Cancel / Deactivate / Test**** a booking. Places are freed, mail stops, the record survives with a reason. Reinstating returns an unpaid booking to Awaiting Payment with a fresh hold, and a paid one to Paid — it never asks a member to pay twice
- ****Block / readmit**** a person, which revokes their ticket without touching their payment

- ****Refunds**** through Razorpay, bounded by what the booking actually paid rather than the event's current fee
- ****Exports**** — see below
- ****Reminder schedule**** per event, plus a manual run
- ****Mailer diagnostics**** — configuration and the last send error
- ****Member directory**** with role promotion, and a ****database browser**** for direct table access
- ****Financial overview**** on the dashboard

Exports

All three are real server-generated `.xlsx` (ExcelJS), sharing one builder so they cannot drift:

| Export | Where | Contents |

|---|---|---|

| Event roster | Manage events → roster → Export | One row ****per participant****: name, mobile, type, booking details, questionnaire answers |

| All rosters | Dashboard → Export all | The same, prefixed with Event and Event date |

| Member list | Members → Export Excel | Every member: contact details, role, verification, events attended, total paid |

Timestamps are IST. There is no CSV anywhere in the application.

Community

Forum posts (admin-authored) with member comments; polls with single-vote enforcement and admin analytics; a photo gallery; event results and personal bests; post-event feedback; attendance streaks; Apple Health workout import.

6. API reference

Everything is under ``/api``. ``GET /health`` is the unauthenticated health check.

Auth is a bearer JWT: ``Authorization: Bearer <token>``. ``authenticateJWT`` runs globally and **lets anonymous requests through** — each route decides for itself. "member+" below means any signed-in role.

Auth — ``/api/auth``

Method	Path	Access
--------	------	--------

---	---	---
-----	-----	-----

POST	<code>`/register`</code>	public
------	--------------------------	--------

POST	<code>`/login`</code>	public
------	-----------------------	--------

POST	<code>`/forgot-password`</code>	public
------	---------------------------------	--------

GET	<code>`/reset-password/:token`</code>	public
-----	---------------------------------------	--------

POST	<code>`/reset-password`</code>	public
------	--------------------------------	--------

GET	<code>`/me`</code>	signed in
-----	--------------------	-----------

| PATCH | ``/me`` | signed in |

| PATCH | ``/me/email`` | signed in |

| POST | ``/me/password`` | signed in |

Email verification — ``/apl/verify``

``GET /status`` · ``POST /email/send`` · ``POST /email/confirm``

Events — ``/apl/events``

| Method | Path | Access |

|---|---|---|

| GET | ``/`` | public (published only for non-admins) |

| GET | ``/:id`` | public |

| POST | ``/`` | ADMIN |

| PUT | ``/:id`` | ADMIN |

| DELETE | ``/:id`` | ADMIN |

| POST | ``/:id/register`` | member+, verified |

| GET | ``/me/registrations`` | member+ |

| GET | ``/registration/:id/ticket`` | owner or ADMIN |

Payments — ``/apl/payments``

| Method | Path | Access | Purpose |

|---|---|---|---|

| GET | ``/config`` | public | Publishable key and mock-mode flag |

| POST | ``/webhook`` | Razorpay | Signature-verified ``payment.captured`` /
``order.paid`` |

POST	``/verify``	member+	Verifies a Checkout callback
POST	``/reconcile/:registrationId``	owner or ADMIN	Asks Razorpay directly whether a booking is already paid
POST	``/reconcile``	ADMIN	The same across every stuck booking
POST	``/order/:registrationId/refresh``	owner or ADMIN	Re-mints an unusable order
POST	``/refund/:registrationId``	ADMIN	
POST	``/simulate``	member+	****Dev only**** — refused in production and with real keys

Admin — ``/api/admin`` (all ADMIN)

``GET /financial-overview`` · ``GET /events/:id/registrations`` ·
``GET /events/:id/roster/export`` · ``GET /rosters/export`` ·
``GET /members/export`` · ``PUT /registrations/:id/block`` ·
``PUT /registrations/:id/status`` · ``POST /holds/sweep`` ·
``GET /events/:id/reminders`` · ``POST /events/:id/reminders/run`` ·
``GET /mailer`` · ``POST /mailer/test`` · ``GET /members`` ·
``PUT /members/:id/role`` · ``GET /polls/:id/analytics``

Race day — ``/api/raceday``

``POST /check-in`` · ``POST /guests/:guestId/admit`` ·
``POST /guests/:guestId/unadmit`` · ``POST /check-in/:registrationId/undo`` ·
``GET /events/:id/dashboard`` · shifts (``GET`` / ``POST`` / ``DELETE``, claim, release) ·
checkpoints (``GET`` / ``POST`` / ``DELETE``, pass). Shift and checkpoint creation is ADMIN; the rest is member+.

Others

- **`**`/api/results`**`** — event results (ADMIN writes), ``/me``, feedback, streaks
- **`**`/api/forum`**`** — posts (ADMIN creates), comments, notifications
- **`**`/api/polls`**`** — list, vote, create and activate (ADMIN)
- **`**`/api/content`**`** — gallery, club info, collaborators, founders, event GPX, video upload signing
- **`**`/api/db`**`** — direct table browse and edit (ADMIN)
- **`**`/api/health`**`** — Apple Health workout import
- **`**`/api/cron/reminders`**`**, **`**`/api/cron/holds`**`** — see §7

7. Scheduled Jobs

Two sweeps. Both are idempotent and safe to call repeatedly.

Endpoint	Job	Configured schedule
----------	-----	---------------------

---	---	---
-----	-----	-----

<code>`/api/cron/reminders`</code>	Sends due event reminders	<code>`30 1 * * *`</code> — 01:30 UTC, 07:00 IST
------------------------------------	---------------------------	--

<code>`/api/cron/holds`</code>	Expires lapsed unpaid holds, sends awaiting-payment nudges	<code>`0 2 * * *`</code> — 02:00 UTC, 07:30 IST
--------------------------------	--	---

Both are gated on ``CRON_SECRET``, accepted as a bearer token or ``?key=``.

The Hobby-plan limitation, stated plainly

Vercel's Hobby plan runs a cron ****once a day****. That is not enough for either job to be accurate:

- Reminder offsets go down to 1 hour, so a "2 hours before" reminder cannot possibly fire 2 hours before.
- A 24-hour hold checked daily expires somewhere between 24 and 48 hours.

Two mitigations are already in the code. Reminders have a ****6-hour grace window**** — a sweep delayed by a normal outage still delivers, but one that missed by days stays quiet rather than emailing "starts in 1 week" ninety minutes before the gun. And capacity treats a lapsed hold as released at read time, so availability is right regardless of the sweep.

****The proper fix**** is an external scheduler hitting both endpoints every 10–15 minutes, which costs nothing:

https://b2runclub-backend-rho.vercel.app/api/cron/reminders?key=\$CRON_SECRET

https://b2runclub-backend-rho.vercel.app/api/cron/holds?key=\$CRON_SECRET

cron-job.org and UptimeRobot both do this. The query-parameter form exists precisely for schedulers that cannot set a header.

Locally, in-process `setInterval` schedulers run instead — reminders every minute, holds every five.

8. File storage

Three drivers, chosen at runtime from the environment, so one build runs anywhere. See `lutils/storage.ts` ([runclub-backend/src/utils/storage.ts](#)).

Priority	Driver	Active when
---	---	---
1	Supabase Storage	<code>`SUPABASE_URL`</code> and <code>`SUPABASE_SERVICE_ROLE_KEY`</code> set
2	Vercel Blob	<code>`BLOB_READ_WRITE_TOKEN`</code> set and Supabase is not
3	Local disk	Neither — writes under <code>`UPLOAD_DIR`</code> , serves <code>`/uploads/*`</code>

****Serverless requires driver 1 or 2.**** The filesystem is read-only apart from ``/tmp``, and ``/tmp`` is not shared between invocations, so a file written during an upload has vanished before anyone can fetch it.

Only URLs are stored, so switching drivers needs no schema change — but existing

rows keep whichever form they were written with, which is why the read and delete helpers accept every shape.

Supabase is preferred simply because the database already lives there, keeping files and rows with one provider.

9. Email

Plain SMTP through Nodemailer, so the provider is a configuration choice rather than a code change. Currently Gmail.

Messages sent: email verification codes, password reset links, event reminders, awaiting-payment nudges, and an admin test message.

The Gmail App Password trap

`SMTP\_PASS` must be a ****16-character Gmail App Password****, because Gmail refuses account passwords over SMTP. And:

> ****Google Invalidates every App Password when the account password changes.****

When that happens, every email silently fails. Nothing else breaks, so the application looks healthy while ****no new member can verify their address, and therefore nobody new can register for an event****. This has already happened once to this club.

The failure is no longer silent: an authentication error is detected by name, logged with the remedy, and surfaced at `GET /api/admin/mailer` as `last_error` with a hint. If members stop receiving mail, look there first.

****To fix:**** generate a new App Password at

[myaccount.google.com/apppasswords](<https://myaccount.google.com/apppasswords>)

and update `SMTP_PASS` in the backend's Vercel environment, then redeploy.

****The durable fix**** is to stop depending on a human-managed mailbox credential at all — move to an API-key transport (Resend, SES, Postmark), which survives password changes and gives delivery logs. Only `mailer.ts` and four environment variables would change.

10. Payments

The flow

1. A member registers. The backend prices the party, creates a **Razorpay order** for the total, and writes the booking as **PENDING** with a hold deadline.
2. The browser opens **Razorpay Checkout** with that order id.
3. On success Razorpay hands the browser `{ order_id, payment_id, signature }`, which the app posts to **/api/payments/verify**. The server recomputes the HMAC-SHA256 of `order_id|payment_id` with **RAZORPAY_KEY_SECRET** and, if it matches, marks the booking **PAID**.
4. Independently, Razorpay calls the **webhook** with `payment.captured` / `order.paid`. The signature is verified against **RAZORPAY_WEBHOOK_SECRET**.

Every path is idempotent, so a repeated confirmation cannot issue a second ticket or take a second payment.

The failure this was built to survive

The browser callback in step 3 is easily lost. A UPI payment on a phone hands off to GPay or PhonePe, and Android frequently evicts the browser tab that has to run the callback. If the webhook is not configured, **the money is captured and the booking stays PENDING for ever** — and Razorpay refuses a second payment against an order it considers paid, so the member cannot even retry.

Two defences:

- **`POST /api/payments/reconcile`** asks Razorpay directly whether a booking is already paid, and settles it if so. The member's own "Pay" button calls this *before* opening Checkout, so an already-paid booking releases its ticket instead of dead-ending. An admin sweep (`POST /api/payments/reconcile`) does the same across every stuck booking. `authorized`-but-uncaptured is reported separately rather than treated as paid.

- **A late payment is honoured.** An `EXPIRED` booking still accepts payment, because refusing a ticket to somebody whose money has been taken is the one outcome that cannot be defended. `CANCELLED`, `DEACTIVATED` and `TEST` do not accept payment — those are deliberate decisions.

Webhook setup — required

In the **Razorpay dashboard** → **Settings** → **Webhooks**, in **Live** mode:

|||

|---|---|

| URL | `https://b2runclub-backend-rho.vercel.app/api/payments/webhook` |

| Events | `payment.captured` **and** `order.paid` |

| Secret | The same value as `RAZORPAY_WEBHOOK_SECRET` |

Without this, the browser is the only thing that can release a ticket.

11. Deployment

Two Vercel projects from the same repository, each with its own root directory.

Backend project

Setting	Value

Root directory	<code>`runclub-backend`</code>
Install	<code>`npm install`</code>
Build	<code>`npm run build && npx prisma migrate deploy && node dist/seed.js`</code>
Entrypoint	<code>`api/index.ts`</code> , with every path rewritten to <code>`/api`</code>

****Migrations and the seed run in the build command****, which is unusual and deliberate: ``npm start`` is never executed on Vercel, so without this the database would never gain a table and no admin would exist.

Both steps are safe to repeat — ``migrate deploy`` applies only pending migrations, and the seed exits early once an admin exists.

>  ****Preview deployments build too****, against whatever ``DATABASE_URL`` is set > for the Preview environment. Give previews their own database, or ****they will migrate production****.

Frontend project

Setting	Value
Root directory	<code>`runclub-frontend`</code>
Framework	Vite
Build	<code>`npm run build`</code> (<code>`tsc -b && vite build`</code>)
Output	<code>`dist`</code>

Its ``vercel.json`` proxies ``/api/*``, ``/uploads/*`` and ``/health`` to the backend and falls back to ``index.html`` for client-side routes.

First-time setup, In order

1. **Supabase** — create the project. From Settings → Database take the **pooler** URI (port 6543) for ``DATABASE_URL`` and the **direct** URI (port 5432) for ``DIRECT_URL``. Create a Storage bucket named ``uploads``.
2. **Backend project** — set root directory, add every variable from §3, deploy. The build runs the migrations and creates the admin.
3. **Frontend project** — set root directory, point the three rewrites in ``vercel.json`` at the backend's URL, deploy.
4. **Domain** — add ``b2club.in`` and ``www.b2club.in`` to the *frontend* project.
5. **Razorpay** — register the webhook (§10).
6. **External cron** — point a scheduler at both ``/api/cron/*`` endpoints (§7).
7. **Verify** — ``GET /health`` returns ``{"status":"healthy","database":"connected"}``; ``GET /api/payments/config`` shows ``mock_mode:false`` and the expected key id;

Admin → Mailer shows no ``last_error`` ; send a test email.

Routine deploys

Push to the default branch. Both projects rebuild. A schema change means adding a migration under ``prisma/migrations`` — write the SQL by hand, matching the numbered convention, and keep it **additive** (nullable columns or columns with defaults) so a deploy never needs a backfill and never breaks a running release.

Rollback

Vercel → Deployments → promote a previous one. **Note that this does not roll back the database.** A migration that has run stays run, which is the main reason to keep them additive.

12. Local development

```
```bash
```

```
Backend — http://localhost:3000
```

```
cd runclub-backend
```

```
npm install
```

```
npm run dev
```

```
Frontend — http://localhost:5173
```

```
cd runclub-frontend
```

```
npm install
```

```
npm run dev
```

```

```

Vite proxies `/api`, `/health` and `/uploads` to port 3000, so the local setup matches production's same-origin arrangement.

### ### Local runs on SQLite, production on Postgres

This is the one genuinely awkward part of the repo. `schema.prisma` declares `postgresql`, because that is the deploy target. Prisma refuses to pair the libSQL adapter with a `postgres` provider, so the client used locally must be generated from a `sqlite-provider schema`.

`npm run dev` therefore:

1. runs `scripts/local-schema.mjs`, which derives `prisma/schema.local.prisma` from `schema.prisma` with the provider swapped to `sqlite`;
2. generates the client from that derived schema;
3. starts the server with `tsx`.

`schema.local.prisma` is `generated, not committed`. Edit `schema.prisma` only — there is exactly one schema to maintain.

> ⚠️ **\*\*A bare `prisma generate` breaks local development\*\*, because it  
> regenerates the client for Postgres. `npm run build` and `postinstall` both do  
> this. Run `npm run dev` (or `npm run generate:local`) afterwards to restore it.**

Because migrations are Postgres SQL, applying a schema change to the local SQLite database means running the equivalent `ALTER TABLE` by hand. SQLite will not accept `NOT NULL DEFAULT CURRENT_TIMESTAMP` — add the column with a constant default, then backfill.

Back up `dev.db` before schema work. It holds real data.

---

## ## 13. Testing

**\*\*`[runclub-backend/scripts/api-test-suite.mjs]``[runclub-backend/scripts/api-test-suite.mjs]`\*\***

— 90 automated API assertions covering registration, validation, pricing, holds, expiry, statuses, capacity, exports and access control. Run against a local server with `node scripts/api-test-suite.mjs`. It seeds its own data; clean up the `Suite %` events and `suite.%` users afterwards.

**- \*\*`npm run test-api`\*\* — an older end-to-end script requiring a built `dist`.**

**Not covered automatically, and needing a human: anything in a browser (the password toggle, radio buttons, responsive layout), real card payments, email delivery, and QR scanning.**

**A manual UAT checklist covering those (228 cases across signup, registration, payments, holds, tickets, the questionnaire, email, admin tools, timezone, access control and devices) was written but is **\*\*not in the repository\*\*** — it was never committed and has since been deleted. Regenerate it if a structured UAT pass is needed; the automated suite above covers the logic layer but none of the browser, payment or email paths.**

---